MSDM5004 Spring 2026
Homework 2 Solution

(Non-exact results are expressed in either the long or the short format of MATLAB.)

1. Iterative methods for solving linear systems I

(1) The Jacobi method solves

$$\begin{cases} x_1^{(k)} &= \frac{1}{8}\left[3x_2^{(k-1)} - 2x_3^{(k-1)} + 20\right] \\ x_2^{(k)} &= \frac{1}{11}\left[-4x_1^{(k-1)} + x_3^{(k-1)} + 33\right] \\ x_3^{(k)} &= \frac{1}{12}\left[-6x_1^{(k-1)} - 3x_2^{(k-1)} + 36\right] \end{cases}.$$

Starting from $\mathbf{x}^{(0)} = \mathbf{0}$,

$$\begin{cases} x_1^{(1)} &= \frac{1}{8}(3\times 0 - 2\times 0 + 20) &= \frac{5}{2} &= 2.5 \\ x_2^{(1)} &= \frac{1}{11}(-4\times 0 + 0 + 33) &= 3 &= 3 \\ x_3^{(1)} &= \frac{1}{12}(-6\times 0 - 3\times 0 + 36) &= 3 &= 3 \end{cases},$$

$$\begin{cases} x_1^{(1)} &= \frac{1}{8}(3\times 3 - 2\times 3 + 20) &= \frac{23}{8} &= 2.875 \\ x_2^{(1)} &= \frac{1}{11}\left(-4\times\frac{5}{2} + 3 + 33\right) &= \frac{26}{11} &= 2.3636 \\ x_3^{(1)} &= \frac{1}{12}\left(-6\times\frac{5}{2} - 3\times 3 + 36\right) &= 1 &= 1 \end{cases}.$$

(2) The Gauss-Seidel method solves

$$\begin{cases} x_1^{(k)} &= \frac{1}{8}\left[3x_2^{(k-1)} - 2x_3^{(k-1)} + 20\right] \\ x_2^{(k)} &= \frac{1}{11}\left[-4x_1^{(k)} + x_3^{(k-1)} + 33\right] \\ x_3^{(k)} &= \frac{1}{12}\left[-6x_1^{(k)} - 3x_2^{(k)} + 36\right] \end{cases}.$$

Starting from $\mathbf{x}^{(0)} = \mathbf{0}$,

$$\begin{cases} x_1^{(1)} &= \frac{1}{8}(3\times 0 - 2\times 0 + 20) &= \frac{5}{2} &= 2.5 \\ x_2^{(1)} &= \frac{1}{11}\left(-4\times\frac{5}{2} + 0 + 33\right) &= \frac{23}{11} &= 2.0909 \\ x_3^{(1)} &= \frac{1}{12}\left(-6\times\frac{5}{2} - 3\times\frac{23}{11} + 36\right) &= \frac{27}{22} &= 1.2273 \end{cases},$$

$$\begin{cases} x_1^{(2)} &= \frac{1}{8}\left(3\times\frac{23}{11}-2\times\frac{27}{22}+20\right) &= \frac{131}{44} &= 2.9773 \\ x_2^{(2)} &= \frac{1}{11}\left(-4\times\frac{131}{44}+\frac{27}{22}+33\right) &= \frac{491}{242} &= 2.0289\ . \\ x_3^{(2)} &= \frac{1}{12}\left(-6\times\frac{131}{44}-3\times\frac{491}{242}+36\right) &= \frac{243}{242} &= 1.0041 \end{cases}$$

2. Iterative methods for solving linear systems II

Let $\mathbf{A}=\begin{pmatrix}4&1&1&1\\1&4&1&1\\1&1&4&1\\1&1&1&4\end{pmatrix}=(A_{ij})$ and $\mathbf{b}=\begin{pmatrix}1\\1\\1\\1\end{pmatrix}=(b_j)$ so that the system can be written as $\mathbf{Ax}=\mathbf{b}$. Note that $\left\|\mathbf{x}^{(k)}-\mathbf{x}^{(k-1)}\right\|_\infty=\max_i\left|x_i^{(k)}-x_i^{(k-1)}\right|$.

(1) The general formula of the Jacobi method is

$$\begin{aligned} x_i^{(k)} &= \frac{b_i}{A_{ii}}-\sum_{j\neq i}\frac{A_{ij}}{A_{ii}}\,x_j^{(k-1)} \\ &= \underbrace{\frac{b_i}{A_{ii}}}_{b_i'}-\sum_j\underbrace{\left(\frac{A_{ij}}{A_{ii}}-\delta_{ij}\right)}_{A_{ij}'}x_j^{(k-1)}\ . \end{aligned}$$

where δ_{ij} is the Kronecker delta and forms an identity matrix $\mathbf{I}=(\delta_{ij})$. Hence, using $\mathbf{x}^{(0)}=\mathbf{0}$, it can be implemented in MATLAB as

```
A = [4 1 1 1; 1 4 1 1; 1 1 4 1; 1 1 1 4];
b = [1 1 1 1]';
n = size(b, 1); % number of variables
b_ = b ./ diag(A); % b'
A_ = A ./ diag(A) - eye(n); % A'
xh = [1 1 1 1]; % x^(k-1); just chosen to start the while loop
xk = [0 0 0 0]; % x^(k); the true starting point
while max(abs(xk-xh)) > 1e-5
    xh = xk;
    for i = 1:n
        xk(i) = b_(i) - sum(A_(i, :) .* xh);
    end
end % ends after 37 iterations
xk  % all four entries = 0.142860548261642
```

(2) The general formula of the SOR method is

$$x_i^{(k)} = (1-\omega)x_i^{(k-1)} + \omega\left[\frac{b_i}{A_{ii}} - \sum_{j<i}\frac{A_{ij}}{A_{ii}}\,x_j^{(k)} - \sum_{j>i}\frac{A_{ij}}{A_{ii}}\,x_j^{(k-1)}\right].$$

Algorithmically speaking, however, it can be simplified as

$$x_i^{(k)} \leftarrow (1-\omega)x_i^{(k-1)} + \omega\left[\underbrace{\frac{b_i}{A_{ii}}}_{b_i'} - \sum_j \underbrace{\left(\frac{A_{ij}}{A_{ii}} - \delta_{ij}\right)}_{A_{ij}'}\,x_j^{(k)}\right].$$

This is valid because when $x_i^{(k)}$ is being computed, $x_j^{(k)}$ with $j > i$ has not been updated yet, so $x_j^{(k)} = x_j^{(k-1)}$ in the sum and restores the general formula. Hence, using $\omega = 1.3$ and $\mathbf{x}^{(0)} = \mathbf{0}$, it can be implemented in MATLAB as

```
A = [4 1 1 1; 1 4 1 1; 1 1 4 1; 1 1 1 4];
b = [1 1 1 1]';
n = size(b, 1);
b_ = b ./ diag(A);
A_ = A ./ diag(A) - eye(n);
xh = [1 1 1 1]; % x^(k-1)
xk = [0 0 0 0]; % x^(k)
w = 1.3; % omega
while max(abs(xk-xh)) > 1e-5
    xh = xk;
    for i = 1:n
        xk(i) = (1-w)*xh(i) + w*( b_(i)-sum(A_(i, :).*xk) );
    end
end % ends after 13 iterations
xk  % [0.142857719032848 0.142855583970553
    %  0.142855627501143 0.142857016314788]
```

3. Numerical differentiation

The central difference formula is

$$f'(x) \approx \phi_h(x) = \frac{f(x+h) - f(x-h)}{2h}$$

for some small h. Using $h = 0.1$, $f'(x = 2)$ can be estimated at $\phi_h(x = 2) = 1.848802637959988$ by the following code.

```
f = @(x) exp(x)/x;
phi = @(x, h) ( f(x+h)-f(x-h) )/( 2*h );
h = 0.1;
phi(2, h) % 1.848802637959988
```

The order of convergence of the estimation can be obtained by the following code, which computes the ratio

$$r = \frac{\phi_h(x = 2) - \phi_{h/2}(x = 2)}{\phi_{h/2}(x = 2) - \phi_{h/4}(x = 2)} .$$

```
f = @(x) exp(x)/x;
phi = @(x, h) ( f(x+h)-f(x-h) )/( 2*h );
h = 0.1;
n = 8;
p = zeros(n, 1); % an array of phi
for i = 1:n
    p(i) = phi(2, h); % p(i) uses h = 0.1*2^(i-1)
    h = h / 2;
end
for i = 1:n-2
    r = ( p(i)-p(i+1) )/( p(i+1)-p(i+2) )
end % r = 3.998112659735039, 3.999530481380195, 3.999882764167103,
    %     3.999970612727584, 3.999993382852703, 3.999998235428166
```

The order is 2 since the ratio r remains to be around $4 = 2^2$.

4. Numerical methods for ordinary differential equations

Let $a = 2$, $b = 4$, and

$$f(t, y) = 2t^{-2}(ty - 2y^2) \quad \text{for} \quad t \in [a, b] .$$

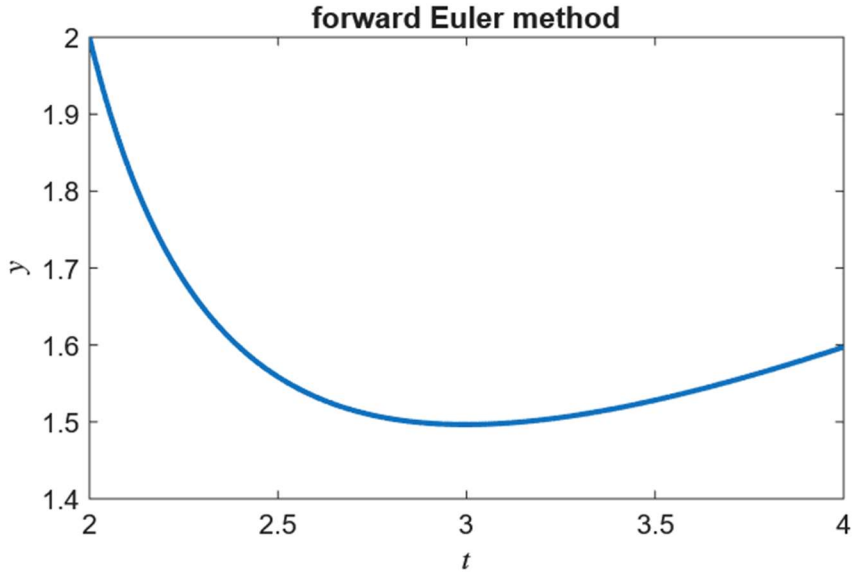
(1) The forward Euler method solves

$$y_{i+1} = y_i + hf(t_i, y_i) .$$

Using $h = \frac{1}{128}$, it can be implemented in MATLAB straightforwardly as

```
a = 2; b = 4; h = 1/128;
n = (b-a)/h; % number of steps
t = linspace(a, b, n+1);
y = 2; % initially; a number is a 1x1 array
f = @(t, y) 2*t^-2 * (t*y - 2*y^2);
for i = 1:n
    y(i+1) = y(i) + h * f(t(i), y(i));
end % finally, y(end) = 1.597282579543474
```

The graph of the solution is



(2) The backward Euler method solves

$$y_{i+1} = y_i + hf(t_{i+1}, y_{i+1}) \ .$$

After some algebra, it can be found that

$$y_{i+1} = \frac{-q \pm \sqrt{q^2 - 4py_i}}{2p}$$

$$\text{for} \quad p = -\frac{4h}{t_{i+1}^2} \quad \text{and} \quad q = \frac{2h}{t_{i+1}} - 1 \ .$$

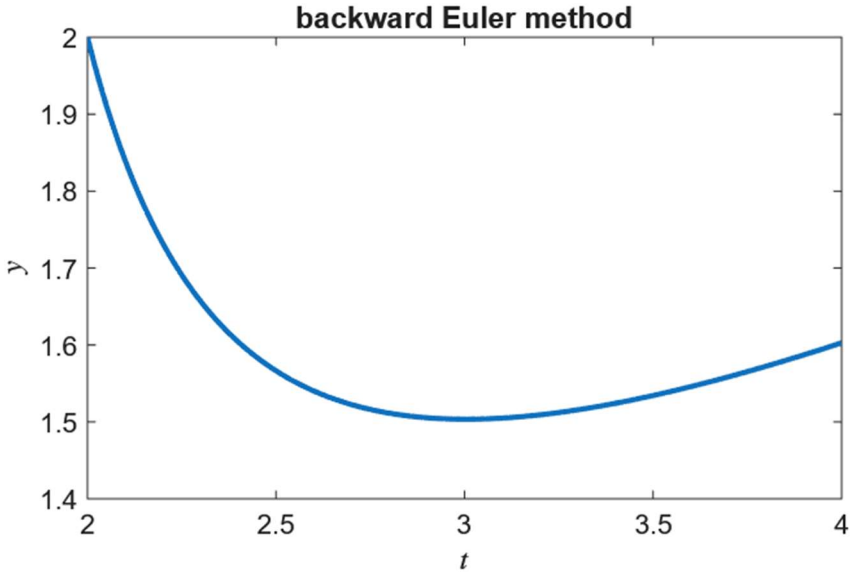
Now check the initial behaviours of the two roots of y_{i+1}: substituting $h = \frac{1}{128}$, $y_1 = y(a) = 2$, and $t_2 = a + h = 2 + \frac{1}{128}$ into the formulae, y_2 becomes

$$\frac{-q + \sqrt{q^2 - 4py_1}}{2p} = -129.9830 \quad \text{or} \quad \frac{-q - \sqrt{q^2 - 4py_1}}{2p} = 1.9849 \ .$$

The former root is rejected because it deviates from y_1 too much and will make y_3 a complex number, so only the latter root is kept. Hence, the backward Euler method can be implemented in MATLAB as

```
a = 2; b = 4; h = 1/128;
n = (b-a)/h;
t = linspace(a, b, n+1);
y = 2; % initially
for i = 1:n
    p = -4*h/t(i+1)^2;
    q = 2*h/t(i+1) - 1;
    y(i+1) = ( -q-sqrt(q^2-4*p*y(i)) )/( 2*p );
end % finally, y(end) = 1.602695688272433
```

The graph of the solution is



5. Numerical methods for partial differential equations (parabolic PDEs)

Remember t is time, x is space, and U_j^n is an approximation to $u(x_j, t_n)$. The truncation error of the given scheme is

$$T(x_j, t_n) = \frac{u(x_j, t_{n+1}) - u(x_j, t_{n-1})}{2\Delta t} - 2\left[\frac{u(x_{j+1}, t_n) - 2u(x_j, t_n) + u(x_{j-1}, t_n)}{(\Delta x)^2}\right].$$

where $\Delta t \equiv t_{n+1} - t_n \equiv t_n - t_{n-1}$ and $\Delta x \equiv x_{j+1} - x_j \equiv x_j - x_{j-1}$.

First perform Taylor expansion on u in time.

$$u(x_j, t_{n\pm1}) = u(x_j, t_n) \pm (\Delta t)\frac{\partial u}{\partial t}\Big|_{(x_j,t_n)} + \frac{(\Delta t)^2}{2}\frac{\partial^2 u}{\partial t^2}\Big|_{(x_j,t_n)} \pm \frac{(\Delta t)^3}{6}\frac{\partial^3 u}{\partial t^3}\Big|_{(x_j,t_n)} + \cdots$$

$$\Rightarrow \quad \frac{u(x_j, t_{n+1}) - u(x_j, t_{n-1})}{2\Delta t} = \frac{\partial u}{\partial t}\Big|_{(x_j,t_n)} + \underbrace{\frac{(\Delta t)^2}{6}\frac{\partial^3 u}{\partial t^3}\Big|_{(x_j,t_n)} + \cdots}_{O[(\Delta t)^2]}$$

Then perform Taylor expansion on u in space.

$$u(x_{j\pm1}, t_n) = u(x_j, t_n) \pm (\Delta x)\frac{\partial u}{\partial x}\Big|_{(x_j,t_n)} + \frac{(\Delta x)^2}{2}\frac{\partial^2 u}{\partial x^2}\Big|_{(x_j,t_n)} \pm \frac{(\Delta x)^3}{6}\frac{\partial^3 u}{\partial x^3}\Big|_{(x_j,t_n)} + \frac{(\Delta x)^4}{24}\frac{\partial^4 u}{\partial x^4}\Big|_{(x_j,t_n)} + \cdots$$

$$\Rightarrow \quad \frac{u(x_{j+1}, t_n) - 2u(x_j, t_n) + u(x_{j-1}, t_n)}{(\Delta x)^2} = \frac{\partial^2 u}{\partial x^2}\Big|_{(x_j,t_n)} + \underbrace{\frac{(\Delta x)^2}{12}\frac{\partial^4 u}{\partial x^4}\Big|_{(x_j,t_n)} + \cdots}_{O[(\Delta x)^2]}$$

Therefore,

$$\begin{aligned} T(x_j, t_n) &= \left(\frac{\partial u}{\partial t}\Big|_{(x_j,t_n)} + O[(\Delta t)^2]\right) - 2\left(\frac{\partial^2 u}{\partial x^2}\Big|_{(x_j,t_n)} + O[(\Delta x)^2]\right) \\ &= \underbrace{\frac{\partial u}{\partial t}\Big|_{(x_j,t_n)} - 2 \cdot \frac{\partial^2 u}{\partial x^2}\Big|_{(x_j,t_n)}}_{0} + O[(\Delta t)^2] + O[(\Delta x)^2] \qquad \left(\because \frac{\partial u}{\partial t} = 2 \cdot \frac{\partial^2 u}{\partial x^2}\right) \\ &= O[(\Delta t)^2] + O[(\Delta x)^2] \ . \end{aligned}$$

It means that the given scheme is second order both in time and space.